Rapport de sécurité — Time Manager

Time Manager — Rapport de sécurité (pentest)

Projet : Time Manager · Ville de Gotham Serveur testé : `http://35.208.70.76` (Google Cloud) Date : 9 octobre 2026

Pour chaque vérification, on répond aux trois questions du sujet : **C'est quoi la faille ? Comment la détecter ? Comment s'en protéger ?**

---

1. Résumé

Trois failles trouvées et deux comportements à risque, sur une application qui résiste bien par ailleurs (injection SQL, base de données, authentification).

#	Vérification	Résultat	Gravité
A	Absence de HTTPS (mot de passe et JWT en clair)	Vulnérable	Haute
B	Pas de limite sur les connexions (force brute)	Vulnérable	Haute
C	Pas de limite de taille des données (DoS)	Vulnérable	Moyenne
D	Back-end (port 4000) exposé en direct	Vulnérable	Moyenne
E	Versions et en-têtes du serveur web	À durcir	Faible
—	Injection SQL	Non vulnérable	—
—	Base de données (PostgreSQL) accessible	Non exposée	—
—	Endpoints accessibles sans connexion	Protégés (401)	—
—	Hash des mots de passe	Solide (bcrypt)	—
—	Cookie JWT lisible en JavaScript	Non (HttpOnly)	—

---

2. Outils utilisés

- **dirb** : recherche de fichiers et de routes cachés sur un serveur web.
- **nmap** : liste les ports ouverts et les logiciels qui tournent derrière.
- **sqlmap** : teste automatiquement les injections SQL.
- **nikto** : scanne un serveur web (en-têtes, versions, fichiers dangereux).
- **curl** : envoi de requêtes manuelles (force brute, taille des données).
- **Wireshark** : lecture du trafic réseau.

---

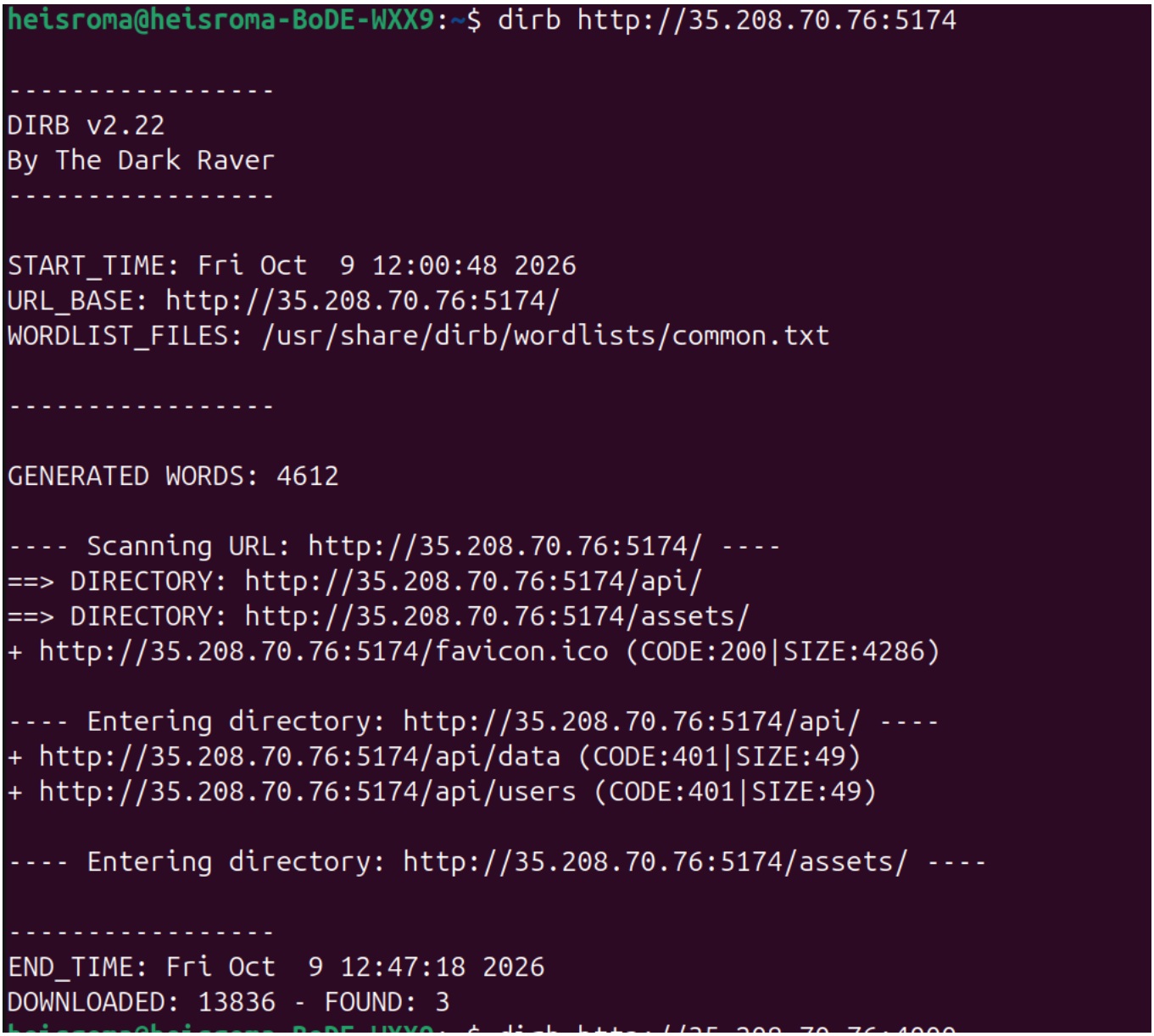
3. Détail des vérifications

Test 1 — dirb : recherche de fichiers et routes cachés

Commande

```
dirb http://35.208.70.76:5174
dirb http://35.208.70.76:4000
```

Résultat



Résultat dirb

- `/api/data` et `/api/users` répondent **401** : les routes existent mais refusent l'accès sans connexion. C'est le bon comportement.
- Aucun fichier sensible trouvé (pas de `/config`, `/.env`, `/.git`, `/admin`).
- `/assets/` et `/favicon.ico` sont publics : normal.

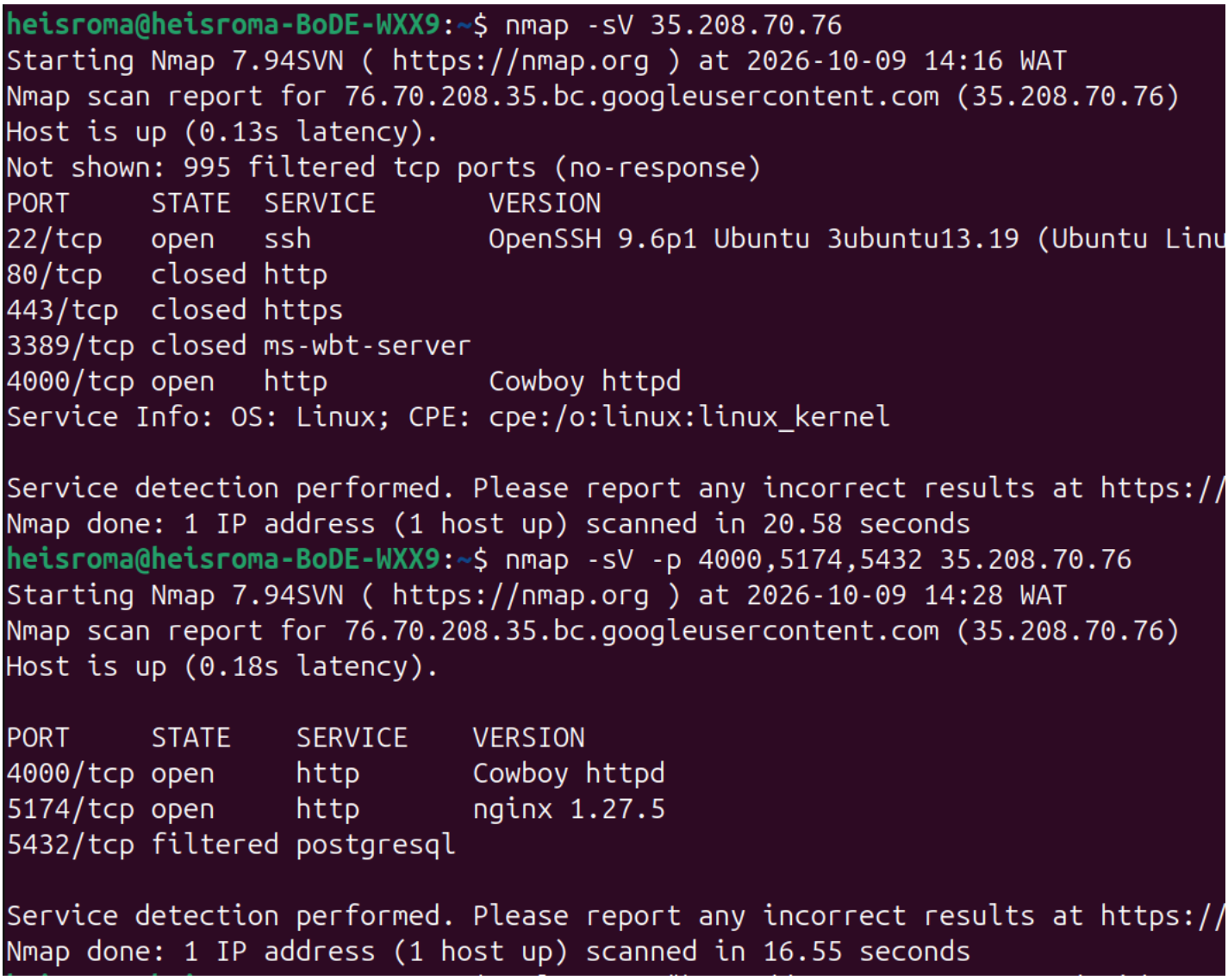
Conclusion : positif. Pas de fichier de configuration exposé, et les endpoints sont protégés. On remarque seulement que le port 4000 répond (voir Test 2).

Test 2 — nmap : ports ouverts et services

Commande

```
nmap -sV 35.208.70.76
nmap -sV -p 4000,5174,5432 35.208.70.76
```

Résultat



Résultat nmap

Port	État	Service	Remarque
22	ouvert	OpenSSH 9.6p1	SSH, version à jour
443	fermé	https	**Aucun HTTPS** (voir faille A)
4000	ouvert	Cowboy httpd	**Back-end exposé en direct** (faille D)
5174	ouvert	nginx 1.27.5	Front, version affichée (faille E)
5432	filtré	postgresql	Base **non accessible** de l'extérieur : bon point

Faille D — Back-end exposé en direct - **Faille :** le back Phoenix (port 4000) est joignable directement depuis Internet, ce qui contourne le reverse proxy Nginx. - **Détection :** `nmap` montre `4000/tcp open http Cowboy httpd`. - **Protection :** fermer le port 4000 au public (pare-feu Google Cloud) et retirer `ports: "4000:4000"` de `docker-compose.prod.yml`, pour que seul Nginx parle au back.

Points positifs : PostgreSQL (5432) est filtré, et SSH est à jour.

Test 3 — sqlmap : injection SQL

Commande

```
sqlmap -u "http://35.208.70.76:4000/api/users/sign_in" \
  --method=POST \
  --data='{"user":{"email":"test@test.com*","password":"test"}}' \
  --headers="Content-Type: application/json" \
  --ignore-code=401 --batch --level=2
```

Résultat



Résultat sqlmap

Environ 379 injections testées (MySQL, PostgreSQL, Oracle, SQLite, boolean, error-based, time-based, UNION). Conclusion de sqlmap :

```
(custom) POST parameter 'JSON #1*' does not seem to be injectable
[CRITICAL] all tested parameters do not appear to be injectable
```

Conclusion : non vulnérable. - **Pourquoi c'est protégé :** le back utilise Ecto, qui fait des requêtes paramétrées. Les entrées ne sont jamais collées dans le texte SQL, donc une quote ou un `OR 1=1` est traité comme du texte, jamais comme du code.

Test 4 — nikto : scan du serveur web

Commande

```
nikto -h http://35.208.70.76:5174
```

Résultat

```
it is recommended to perform only basic UNION tests if there is not at least
reduce the number of requests? [Y/n] Y
[14:55:25] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[14:55:31] [INFO] testing 'MySQL UNION query (NULL) - 1 to 10 columns'
[14:55:36] [WARNING] (custom) POST parameter 'JSON #1*' does not seem to be
[14:55:36] [CRITICAL] all tested parameters do not appear to be injectable.
f you wish to perform more tests. If you suspect that there is some kind of
d try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch
[14:55:36] [WARNING] HTTP error codes detected during run:
401 (Unauthorized) - 379 times
[14:55:36] [WARNING] your sqlmap version is outdated

[*] ending @ 14:55:36 /2026-10-09/

heisroma@heisroma-BoDE-WXX9:~$ nikto -h http://35.208.70.76:5174
- Nikto v2.1.5
---------------------------------------------------------------------------
+ Target IP:          35.208.70.76
+ Target Hostname:    76.70.208.35.bc.googleusercontent.com
+ Target Port:        5174
+ Start Time:         2026-10-09 14:59:04 (GMT1)
---------------------------------------------------------------------------
+ Server: nginx/1.27.5
+ Server leaks inodes via ETags, header found with file /, fields: 0x6ac8efc
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ OSVDB-3092: /css: This might be interesting...
+ 6544 items checked: 0 error(s) and 3 item(s) reported on remote host
+ End Time:           2026-10-09 15:20:04 (GMT1) (1260 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Résultat nikto

6544 points vérifiés, 3 remarques mineures.

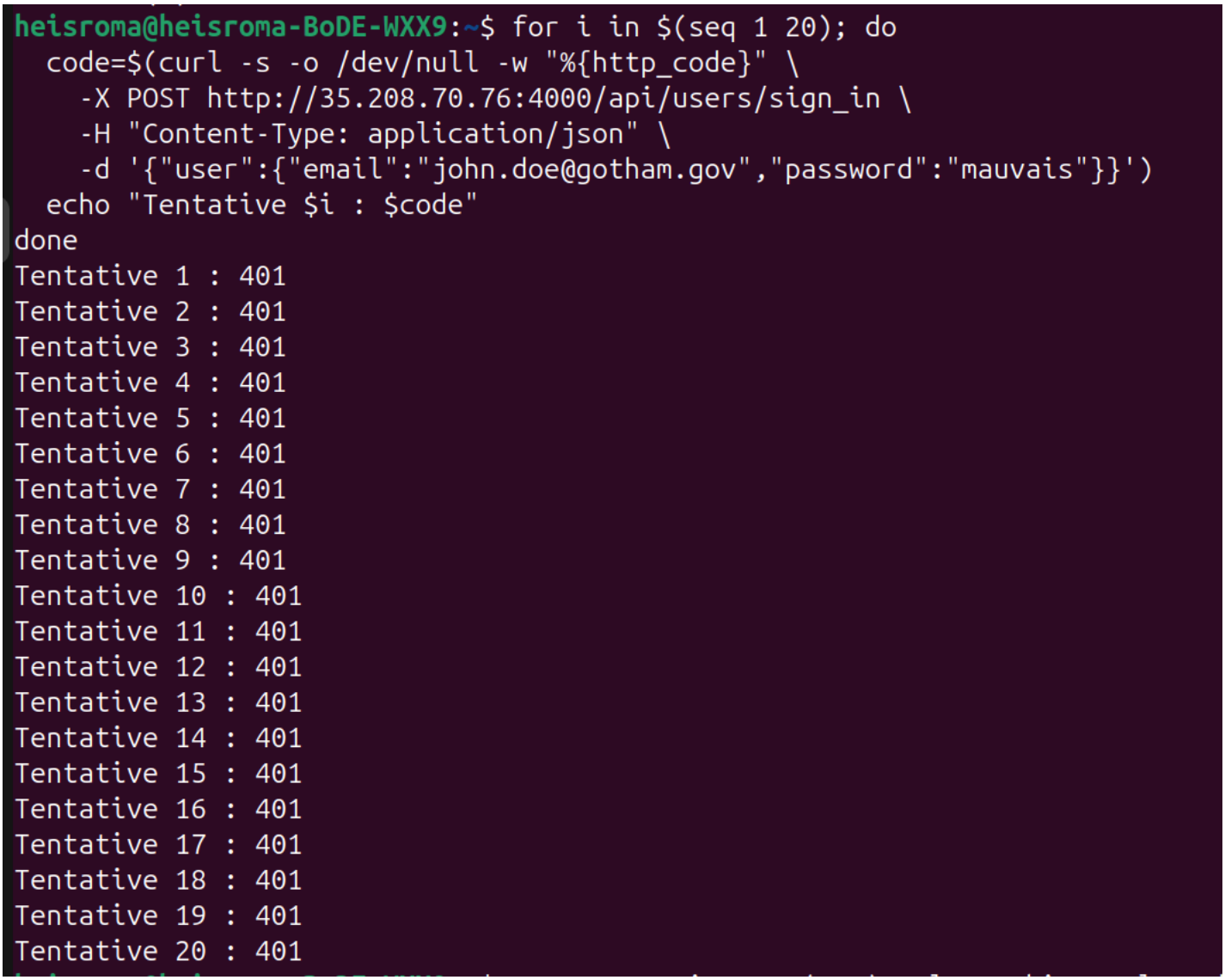
Faille E — Versions et en-têtes du serveur web - Faille : - `Server: nginx/1.27.5` : la version de Nginx est annoncée. - `X-Frame-Options` absent : l'app peut être affichée dans une iframe piégée (clickjacking). - ETags : fuite d'identifiants internes de fichiers (très faible). - **Détection :** `nikto -h http://35.208.70.76:5174`. - **Protection :** dans `nginx.conf`, ajouter : `server_tokens off;  etag off;  add_header X-Frame-Options "SAMEORIGIN";  add_header X-Content-Type-Options "nosniff";`

Test 5 — Force brute sur la connexion

Commande

```
for i in $(seq 1 20); do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST http://35.208.70.76:4000/api/users/sign_in \
    -H "Content-Type: application/json" \
    -d '{"user":{"email":"john.doe@gotham.gov","password":"mauvais"}}'
done
```

Résultat



Résultat force brute

Les 20 tentatives renvoient `401`. Aucune n'est bloquée.

Faille B — Pas de limite sur les connexions - **Faille :** rien ne limite le nombre de tentatives de connexion. Un attaquant peut tester des milliers de mots de passe sans être bloqué. - **Détection :** 20 connexions ratées d'affilée, toutes acceptées (`401`), jamais de `429 (Too Many Requests)`. - **Protection :** limiter le nombre de requêtes sur `/api/users/sign_in` (par exemple 5 tentatives par minute et par IP), avec un plug de rate limiting ou une règle Nginx (`limit_req`).

Test 6 — Déni de service par la taille des données

Commande (après connexion, envoi d'un corps de ~5 Mo)

```
curl -b cookie.jar -X PUT http://35.208.70.76:4000/api/data/absences \
  -H "Content-Type: application/json" \
  -H "x-xsrf-token: <token>" \
  --data @big.json      # fichier d'environ 5 Mo
```

Résultat : le serveur répond `200`. Les 5 Mo sont acceptés et enregistrés.

Faille C — Pas de limite de taille des données - **Faille :** l'endpoint `/api/data` accepte une donnée énorme sans limite. Un utilisateur connecté peut remplir la base et ralentir l'affichage pour tout le monde. - **Détection :** `PUT /api/data/absences` avec un corps de 5 Mo renvoie `200`. - **Protection :** limiter la taille du corps des requêtes (option `:length` du parser JSON de Phoenix) et plafonner la taille de chaque donnée partagée côté `SharedData`.

Test 7 — HTTPS et mots de passe en clair (Wireshark)

Méthode

Capture du trafic avec Wireshark, filtre `http and ip.addr == 35.208.70.76`, puis connexion depuis le navigateur. La requête `POST /api/users/sign_in` montre, en clair :

```
{"user":{"email":"...","password":"..."}}
```

et la réponse contient le cookie `set-cookie: time_manager_token=...` (le JWT), également lisible.

Faille A — Absence de HTTPS (faille la plus grave) - **Faille :** l'application est en HTTP simple. Le mot de passe (à la connexion) et le cookie JWT (à chaque requête) circulent en clair. - **Détection :** Wireshark montre le mot de passe et le token en clair ; `nmap` confirme que le port 443 (HTTPS) est fermé. - **Protection :** - mettre un certificat **HTTPS** (Let's Encrypt) devant l'application ; - rediriger tout le trafic HTTP vers HTTPS ; - passer le cookie en `secure: true` pour qu'il ne parte jamais hors HTTPS. - **Pourquoi c'est grave :** une personne sur le même réseau Wi-Fi peut lire le mot de passe, ou voler le cookie JWT et se connecter à la place de l'utilisateur sans connaître son mot de passe.

Point positif : le cookie est en **HttpOnly**, donc il n'est pas lisible en JavaScript (protection contre le vol de token par du code injecté). Le problème vient uniquement de l'absence de HTTPS.

---

4. Plan de correction (par ordre de priorité)

1. **HTTPS** (faille A) : certificat Let's Encrypt, redirection HTTP→HTTPS, cookie `secure: true`. Corrige aussi les mots de passe en clair.
2. **Limiter les connexions** (faille B) : rate limiting sur `/api/users/sign_in`.
3. **Limiter la taille des données** (faille C) : taille max du corps des requêtes et des données partagées.
4. **Fermer le port 4000** (faille D) : pare-feu + retrait du port dans `docker-compose.prod.yml`.
5. **Durcir Nginx** (faille E) : `server_tokens off`, en-têtes de sécurité.
6. **Divers** : refuser de démarrer en production sans `JWT_SECRET` défini, et masquer `/swagger` et `/api/openapi` en production.

---

5. Conclusion

L'application est bien protégée sur les points les plus souvent exploités : injection SQL, accès à la base de données, protection des endpoints et hachage des mots de passe. Les failles restantes portent surtout sur la configuration du serveur (HTTPS, exposition des ports, limites de requêtes) et sont toutes corrigeables sans refonte du code.